NANYANG TECHNOLOGICAL UNIVERSITY | SINGAPORE

PE6203 • Generative AI and Agentic AI

Cost-Aware Harness Optimization

Final Project
Group B3

SUBMITTED BY

1. ROHIT PANDA G2606819F	2. GHIA ISHA KIRTI G2609983K
3. KAIVELIKKAL ABIN KRISHNA G2606565B	4. SYEDYASEEN ROSHAN TUSHAR G2602441D
5. UBAIDULLA ASMITHA G2602440G	6. ULFA HERDYANI G2609129K

LECTURER	SUBMISSION DATE	ACADEMIC YEAR
Bo An	1 November 2026	2026 / 2027

Executive Summary

We evaluated whether harness engineering can improve a fixed cheap language model enough to approach a stronger model on containerized terminal tasks. Our modular harness extends mini-swe-agent and integrates with Harbor. Six independently switchable components address task instructions, context management, tool use, budgets, failure recovery and submission checking. The final experiment covers 11 configurations on a frozen 40-task TUA-Bench subset. One attempt per configuration and task produced 438 scored trials from 440 planned trials. Two H0+C0 trials failed during Docker setup. [1-4]

The full harness on GPT-OSS-20b, V, reaches mean verifier reward 0.164, compared with 0.089 for the stock harness, B0. The paired difference is +0.075 with a 95% bootstrap interval of +0.008 to +0.158. V approaches the expensive stock baseline E at 0.169, but the interval for V minus E spans zero and does not establish statistical equivalence. Known inference cost per 40-task run is at least $0.0474 for V and $0.0558 for E. Unknown usage makes the former a lower bound, so the exact cost advantage remains unverified. On the project's central question, the cheap model with our harness therefore did not exceed the expensive model with the default harness on mean reward (0.164 against 0.169); its lower recorded cost is itself only a lower bound. The full harness with GPT-OSS-120b, V-E, obtains the highest mean reward, 0.260, at known cost of at least $0.0919. [2-5]

The task-appropriate prompt, C0, provides the clearest standalone gain on its 38 paired tasks. Tool-interface changes have a similar point estimate but substantially higher cost. Other components do not show a clear standalone benefit. Tool-call format failures dominate the recorded failure classes, while our own validation rules also introduce false positives. We preserve the evaluated harness-v1 results separately from subsequent harness-v1.1 fixes.

1. Introduction

An agent harness determines how a model receives instructions, accesses tools, observes results and decides when to stop. These decisions can change task outcomes even when the underlying model remains fixed. A more capable model can still fail if it answers in plain text when the runner requires a tool call. Conversely, a compact prompt or a better tool interface may let a cheaper model execute a useful

plan. The project therefore evaluates performance jointly with inference spending rather than rewarding accuracy alone. [1]

Our central question is whether a cheap model with an improved harness can approach an expensive model using the default harness at lower runtime cost. We study GPT-OSS-20b as the fixed base model and GPT-OSS-120b as the stronger comparator. We evaluate a full harness, an unoptimized port of the loop, and six standalone component additions. This design distinguishes changes due to the execution loop from changes due to individual components.

The contribution is an implemented and observable harness, a reproducible fixed-subset experiment, explicit treatment of unknown token usage, and a failure analysis tied to saved trajectories. The results support a reward improvement over the cheap stock baseline. They also show that adding controls can increase spending or reject valid work. We treat the near match with the expensive baseline as an observed result on this subset, not a claim that the cheap model has become generally equivalent to the stronger one.

2. Experimental setup

2.1 Benchmark and split

TUA-Bench contains 120 terminal tasks graded by execution-based verifiers. We use commit 3497fd320abcafaf4797424192c891a593fd7964. The frozen eval subset contains eight tasks from each of five families: Office and Productivity, Web and Information, System and Software Operations, Scientific and Engineering, and Multimedia and Design. Ten separate dev tasks support model pilots and harness development. Hash ranking with seed 42 makes the split reproducible. [1,6]

Before model evaluation, an oracle gate runs reference solutions to identify environment or verifier failures. Four tasks were replaced with the next eligible reserve from the same family. Reference-solution bugs that agents could work around, and continuous metrics that legitimately produce partial credit, remained in the subset. The final amended fingerprint is 973635ea4e6a477e2fbfffd1045494cb69e0678c02ceea2276c56c63fa0468b8. Tasks and official verifiers remain unmodified.

The headline measure gives each sampled task equal weight and therefore each family equal weight. A secondary estimate reweights family means to the

benchmark population: Office 46, Web 22, System 19, Scientific 17 and Multimedia 16 out of 120. This is a projection from the subset, not a measurement on the remaining 80 benchmark tasks.

2.2 Models and selection

Dev pilots compare GPT-OSS-20b on Darkbloom, Qwen3-Coder-30B on SiliconFlow, and GPT-OSS-120b on Venice. Their ten-task mean rewards are 0.200, 0.319 and 0.366 respectively. Qwen submits reliably but costs 25.44 times the 20b pilot. We retain 20b as the cheapest tested base candidate with non-trivial success, and 120b as the expensive comparator. A three-task 20b provider diagnostic does not establish a reward advantage from switching to DeepInfra. [9]

Both selected models use OpenRouter, pinned endpoints, disabled fallback, low reasoning effort and max_tokens 8192. Temperature and seed remain unspecified as in the pilots. We hold these settings fixed for every harness configuration using the same model. C0 intentionally changes the prompt because prompting is part of the harness intervention. Frozen input/output prices per million tokens are $0.018/$0.09 for 20b and $0.03/$0.15 for 120b. Appendix B gives the accounting and limits.

2.3 Configurations and limits

Configuration	Model	Harness condition
B0	20b	Stock mini-swe-agent 2.4.6
E	120b	Stock mini-swe-agent 2.4.6
H0	20b	Our execution loop with all components off
H0+C0 to H0+C5	20b	One component added to H0 at a time
V	20b	Full harness with C0 through C5
V-E	120b	Full harness with C0 through C5

Table 1. Evaluated configuration families

Every task has a 2,400-second agent limit. B0, E, H0 and ablations without C3 retain no step or dollar cap. C3, V and V-E have 50 steps and respective task budgets of $0.01, $0.01 and $0.0167. C3 also raises the command timeout from 30 to 180 seconds. It therefore changes execution budgets, not only awareness of an otherwise identical budget. Actual cost differences are reported explicitly. A run means one attempt on each of the 40 eval tasks.

2.4 Execution and reproducibility incidents

The final runs took place on 25-27 October on Windows with WSL2 and Docker, using linux/amd64 containers and one concurrent trial. The published results retain harness-v1 at commit 6c87103. A pre-eval C2 deadlock fix deferred helper upload until first use. A guardrail change made valid sudo commands log-only under U-INT-05. These changes precede the evaluated baseline. [3,10]

B0/E initially failed because an unpinned container dependency resolution imported a missing mcp module. The runner uses UV_EXCLUDE_NEWER=2026-10-13 for B0/E to recover the pilot's dependency snapshot without changing stock agent behavior. Failed infrastructure attempts are archived and excluded from scoring. Subsequent runtime and tooling fixes belong to harness-v1.1 and were not rerun to replace v1 results. Two H0+C0 trials, tasks 004 and 099, remain without rewards after Docker credential errors. Ten configurations have 40 scores; C0 has 38.

3. Harness architecture and observability

Harbor creates a task environment and invokes CostAwareAgent, which constructs the loop, component hooks, gateway and container adapter. The HarnessAgent loop assembles a model-visible context, queries the model, validates the proposed action, executes it in the task container and appends the resulting observation. Hooks run around query, execution and submission. This separation lets the same component operate alone or alongside the others. [6]

The OpenRouter gateway records one calls.jsonl row per HTTP attempt, including retries. A row identifies the configuration, task, trial, model, provider, call purpose, token usage and known table cost. Failed calls with no usage retain null token fields. events.jsonl records validation blocks, format errors, offloads, budget nudges, recovery events and submission checks. The full trajectory preserves the model's actual responses and tool actions, while the independent verifier writes the reward.

The container adapter supplies bounded observations and an environment allow-list. API credentials remain outside the evaluated task container. Each task runs in a disposable environment. The host-side collector reads either the harness call log or the stock agent's trajectory, recomputes cost from the frozen price table and cross-checks the harness metadata in result.json. This provides an independently checkable link between individual calls and aggregate results.

H0 is a control for the ported loop. Its mean reward exceeds B0 by +0.042, with interval -0.008 to +0.108. We report both values and use H0 as the root for component ablations. An interval spanning zero is not a formal equivalence test and does not prove that the two execution paths behave identically.

4. Harness components

4.1 Prompt context and budget

C0 replaces the stock software-issue-oriented prompt with instructions suited to terminal tasks. It explicitly states that only tool calls execute, that each shell action starts fresh, that outputs must use the requested paths, and that completion requires the standalone submission sentinel. The prompt contains no task-specific answers.

C1 offloads observations longer than 2,000 characters to a container file and shows the first and last 800 characters with the file path. Older observations shrink to 300 characters. Its middle-out compaction retains the task and selected exchanges, with a six-exchange tail, when estimated context crosses a threshold capped at 24,000 tokens. No compaction event occurred in eval, so the C1 result primarily measures shortening and offloading rather than the compaction policy. The component cannot be credited with benefits from a mechanism that never triggered. [6,8]

C3 enforces the 50-step and dollar caps, adds budget status to observations, and supplies phase nudges at 70% and 90% of steps. Its command timeout is 180 seconds. The one-cent budget rationale is approximately 50 calls times 11,000 prompt tokens at $0.018 per million, or $0.0099 in input cost. Completion tokens and reservations also count, so this does not guarantee that all 50 steps are affordable. The expensive-model cap scales to $0.0167.

4.2 Tools recovery and submission

C2 adds read_file, write_file, edit_file and search alongside bash. These are generic operations with argument schemas, not benchmark-specific shortcuts. Validation checks malformed commands and submission syntax before execution. Only the first tool call in a response executes. The consecutive format-error allowance rises from three to five. The C2 ablation bundles tool granularity, validation and error handling, so it cannot identify which of these subchanges causes its result.

C4 hashes normalized commands and intervenes after repeated failures. It also checks repeated observations and provides a recovery nudge when progress stalls.

Such interventions aim to stop unproductive loops but can inhibit a useful probe if the environment changes; the post-eval audit identified this as a defect requiring a fix.

C5 intercepts the first submission and checks extracted output paths for existence, non-empty content and modification during the trial. It returns a checklist and asks the agent to inspect the outputs before submitting again. It is intentionally bounded rather than an unlimited gate. This is an artifact check, not a semantic verifier: an existing file may still contain the wrong result. All six components are deterministic and make no model calls of their own.

5. Guardrails and safety

Always-on G rules apply to our harness configurations. They aim to restrict destructive commands, preserve grader-path integrity and log secret probing. C2 adds usability and formatting checks for interactive commands, command size, heredocs and submission. B0 and E retain the unmodified stock harness and do not run these live rules. The official verifier remains independent of the agent's completion claims. [7]

The eval review counted 16 blocks and three sudo logs across the 11 configurations. Thirteen blocks came from U-FMT-03 and all thirteen rejected valid shell syntax. For example, nested heredocs and Ruby's left-shift operator were mistaken for unterminated heredocs. A piped interpreter also triggered a usability false positive. We treat these events as harness defects rather than safety successes. A block preceding failure does not establish that the block was its sole cause.

No always-on G-rule matches appeared in the documented 3,094-command live/offline review. This is evidence about these trajectories, not proof that the guards are complete or resistant to adversarial inputs. The audit found latent rule gaps and overly broad matches that never triggered during eval. Fixes are tracked separately in docs/bug_fixes.md. Appendix C relates these controls to the course concepts of execution authority, sandboxing, validation, stopping and observability.

6. Accuracy cost results

6.1 Headline performance

The benchmark's success metric is mean verifier reward, which can include partial credit. Table 2 uses that definition. Inference cost uses the frozen token table and excludes verifier charges. A greater-than-or-equal sign marks a lower bound, because some HTTP 429 attempts have no reported usage. [2-5]

Config	Mean reward	Family weighted	Known USD per run	Model calls
B0	0.089	0.086	$0.0286	449
H0	0.131	0.145	≥ $0.0336	496
V	0.164	0.143	≥ $0.0474	659
E	0.169	0.116	$0.0558	514
V-E	0.260	0.219	≥ $0.0919	795

Table 2. Headline held out mean rewards and known inference costs

V improves on B0 while spending more than B0. Its advantage is therefore a reward gain at a higher known cost, not a universal reduction in inference spending. V approaches E's reward with lower recorded known cost. Because V's total is incomplete, the exact saving against E cannot be guaranteed. V-E has the largest observed reward and the largest known inference cost among the headline configurations.

Family reweighting changes the reading: V reaches 0.143 and E 0.116, while H0 reaches 0.145. E's positive rewards are concentrated in Scientific and Multimedia tasks. The full harness therefore does not dominate every control on every weighting. No uncertainty interval for the reweighted difference is claimed here.

6.2 Paired comparisons

We bootstrap paired task differences with 10,000 resamples and seed 42. Only tasks scored in both configurations enter a comparison. Intervals are exploratory and are not adjusted for the multiple comparisons in this report.

Comparison	Mean difference	95 percent interval	Wins ties losses
V minus B0	+0.075	+0.008 to +0.158	6 / 33 / 1
V minus E	-0.004	-0.113 to +0.107	5 / 31 / 4
V-E minus B0	+0.170	+0.045 to +0.304	10 / 29 / 1
V-E minus E	+0.091	-0.013 to +0.203	6 / 33 / 1
E minus B0	+0.079	-0.012 to +0.183	5 / 34 / 1

Comparison	Mean difference	95 percent interval	Wins ties losses
H0 minus B0	+0.042	-0.008 to +0.108	3 / 36 / 1

Table 3. Paired mean reward differences and bootstrap intervals

The V-B0 interval excludes zero. Most other headline intervals include both negative and positive effects. In particular, an inconclusive V minus E comparison does not establish statistical equivalence. Ties dominate because many tasks receive zero under both configurations.

6.3 Runtime Pareto frontiers

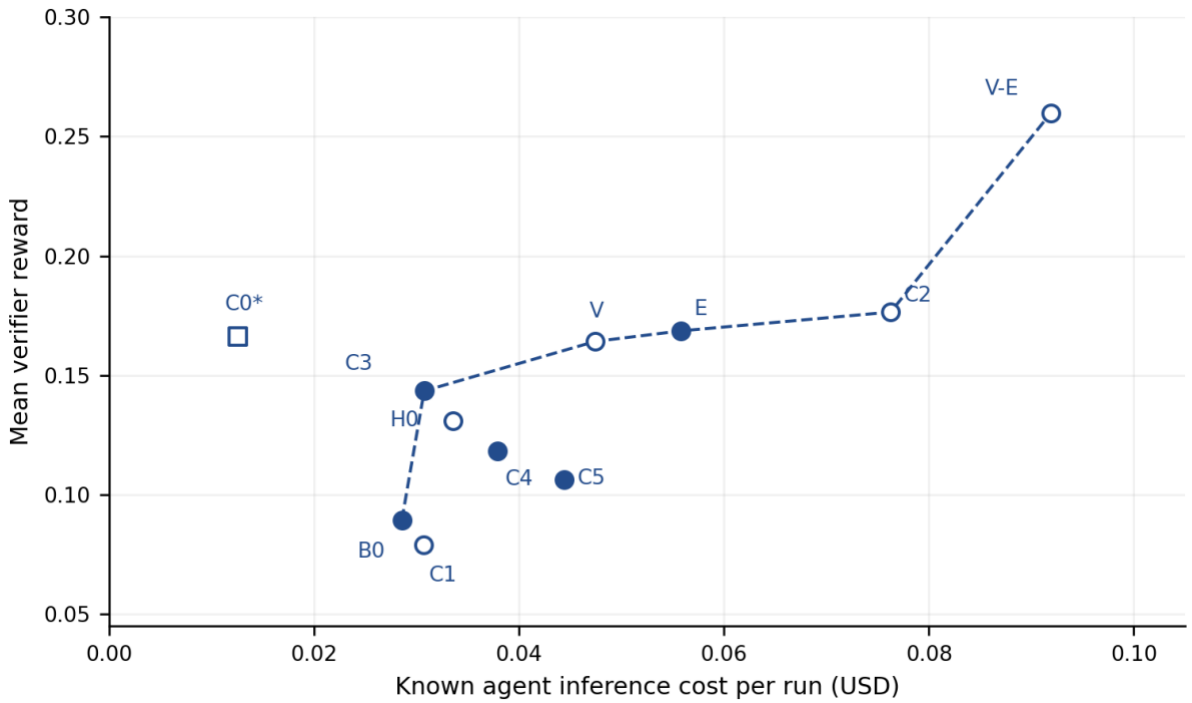


Figure 1. Runtime reward cost frontier admitting known cost lower bounds

Figure 1 plots every evaluated configuration. C0-C5 denote additions to H0. Hollow points have incomplete cost; C0* additionally has only 38 scored tasks. Its $0.0126 represents observed inference on those 38 tasks, not a completed 40-task run. The dashed frontier admits lower bounds and excludes C0 from full-coverage dominance comparisons. Its ordered members are B0, C3, V, E, C2 and V-E.

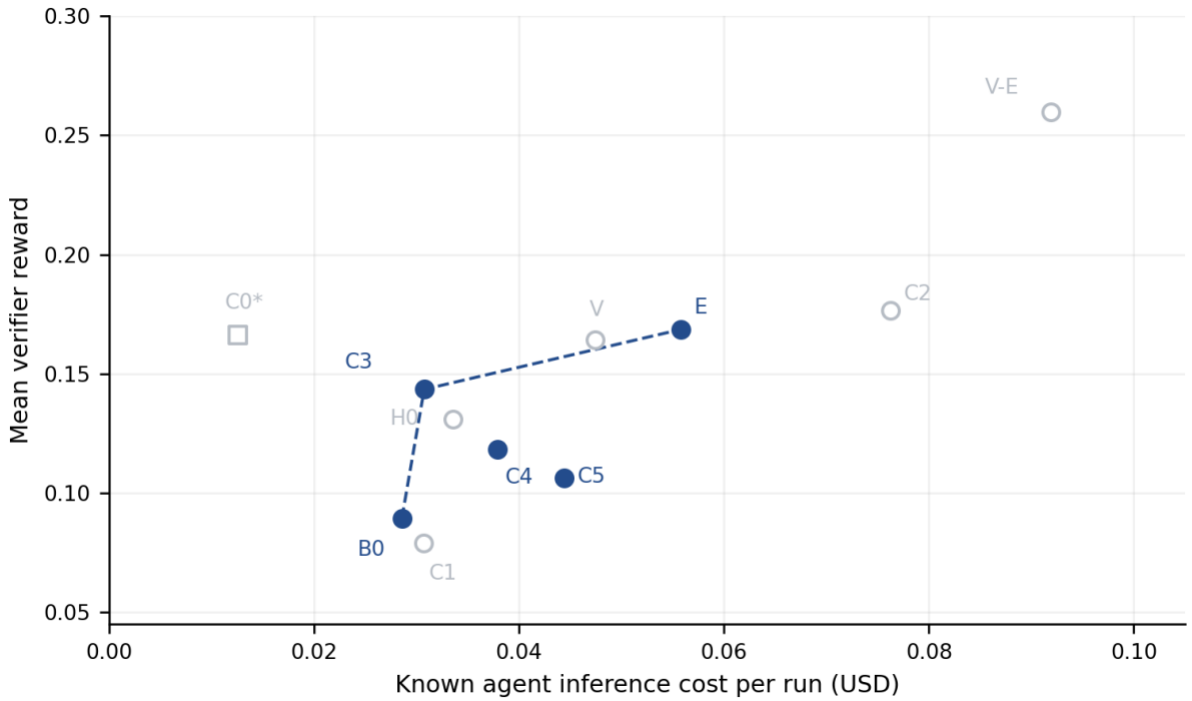


Figure 2. Strict runtime frontier using complete cost observations

The strict frontier in Figure 2 contains B0, C3 and E. Gray points show observations that cannot enter it because cost or reward coverage is incomplete. Reporting both policies makes the consequence of unknown usage explicit. A lower-bound point could move right if missing cost becomes known, so its apparent dominance is conditional.

6.4 Strict single attempt pass rates

To separate full completion from mean reward, we also calculate strict pass@1 as the fraction of tasks with verifier reward exactly 1 after one recorded attempt. This uses the saved trial rewards, not the failure label, and does not estimate performance after retries or repeated samples. [4,8]

Config	Full passes	Scored tasks	Strict pass at 1
B0	1	40	2.5 percent
H0	2	40	5.0 percent
V	3	40	7.5 percent
E	4	40	10.0 percent
V-E	6	40	15.0 percent

Table 4. Strict single attempt full completion rates

The full harness closes much of the mean-reward gap with E, but it still has fewer strict full passes. These are held-out eval results for fixed configurations. Appendix B includes strict outcomes for the remaining configurations.

7. Component ablation and parameter sensitivity

We add one component at a time to H0. This isolates each bundled component as a standalone addition, rather than removing it from V. It is a sensitivity analysis of component presence, not a sweep of numeric thresholds. [5,6]

Component	Mean reward	Difference versus H0	95 percent interval	Known run cost
C0 Prompt	0.166*	+0.046	+0.000 to +0.110	≥ $0.0126*
C1 Context	0.079	-0.052	-0.143 to +0.029	≥ $0.0307
C2 Tools	0.176	+0.046	-0.046 to +0.137	≥ $0.0763
C3 Budget	0.143	+0.013	-0.041 to +0.079	$0.0307
C4 Recovery	0.118	-0.012	-0.092 to +0.063	$0.0379
C5 Submit gate	0.106	-0.024	-0.129 to +0.083	$0.0444

Table 5. Standalone component ablations against H0

*C0 uses 38 paired tasks. H0's mean on those tasks is 0.120, rather than its full-subset 0.131. C0's lower interval endpoint is approximately 0.000098 before rounding. Its 247 calls compare with 470 for H0 on the same 38 tasks. The estimated gain is therefore positive but only narrowly so under this exploratory interval.

C2 has the highest observed mean reward among the fully scored cheap-model configurations, but its known cost exceeds E's and is more than twice H0's. Longer format-error tolerance may keep execution going and increase spending. The experiment cannot separate that effect from the granular tools and validation bundled into C2.

C3 is close to neutral on reward and slightly cheaper on known cost. C1, C4 and C5 have negative point estimates, but their intervals include zero. These do not prove that a component is harmful in every task or setting. No failed trial received the budget_exhausted primary label, and the documented cap hit at V/088 scored 1.0. Numeric sensitivity to the step cap, dollar cap, offload threshold, context threshold or retry schedule remains unmeasured.

Interactions are also unresolved. V gains about +0.033 over H0, less than C0 or C2 alone, while the six standalone deltas sum to roughly +0.016. C0's partial cohort and single-run noise further limit this arithmetic. These observations suggest overlap or interference, not an identified causal interaction. A factorial or leave-one-out study would be needed to distinguish the mechanisms.

8. Cost breakdown and accounting

8.1 Agent inference and unknown usage

We price each call using input and completion tokens at the frozen rates. Reasoning tokens are not added a second time when they are already included in completion usage. Each HTTP attempt remains a distinct record. If either usage field is null, the call contributes to n_unknown_calls and makes usage_complete false. Exact total cost is then unset, while known_cost_table_usd remains the sum of known calls. [4,6]

The collector cross-checks harness totals against agent_result.metadata.known_cost_table_usd in each trial result. A missing log is not evidence of a free run. We retain missing-cost and missing-reward indicators separately. Eighteen unknown-cost attempts occur across the six affected configurations: H0 3, V 2, V-E 7, C0 1, C1 3 and C2 2. The documented events are rate-limit rejections, but no zero charge is assumed without usage evidence.

Config	Prompt tokens	Completion tokens	Known USD per run
B0	1.19 million	79 thousand	$0.0286
H0	1.44 million	84 thousand	≥ $0.0336
V	2.22 million	84 thousand	≥ $0.0474
E	1.56 million	60 thousand	$0.0558
V-E	2.71 million	71 thousand	≥ $0.0919

Table 6. Token usage and inference dollars per run

8.2 Harness overhead

The harness makes no separate summarization, routing or verification model calls, so direct harness-call cost is $0. Prompt overhead still consumes inference tokens. We estimate its share using each call's prompt tokens multiplied by the fraction of prompt characters attributed to harness-injected text. Multiplying the estimated input tokens by the input rate gives injected-prompt cost. This is a character-share estimate, not exact tokenizer attribution.

The estimates are $0.0061 per V run, about 13 percent of known inference cost, and $0.0126 per V-E run, about 14 percent. These amounts are already included in inference cost and must not be added again. Stock format-error feedback is accounted

for separately: approximately $0.0021 in V and $0.0025 in V-E. Table 6A gives the split for every configuration. [2,10]

Config	Known inference cost	Harness-made model calls	Injected prompt text (est.)	Share of known cost
B0	$0.0286	$0	—	—
E	$0.0558	$0	—	—
H0	≥ $0.0336	$0	$0	0%
V	≥ $0.0474	$0	$0.0061	13%
V-E	≥ $0.0919	$0	$0.0126	14%
H0+C0	≥ $0.0126 (38 tasks)	$0	$0	0%
H0+C1	≥ $0.0307	$0	$0.0007	2%
H0+C2	≥ $0.0763	$0	$0.0014	2%
H0+C3	$0.0307	$0	$0.0013	4%
H0+C4	$0.0379	$0	$0.0008	2%
H0+C5	$0.0444	$0	$0.0007	2%

Table 6A. Harness overhead per run, already included in known inference cost. A dash marks the stock baselines, where injected text is not estimated

8.3 Experiment spending and deployment cost

Summing the current summary.csv yields $0.489754008 in known cost across active eval attempts and $0.492205728 when the preserved archived attempts are included. These are derived totals of the eleven configuration rows, not invoices or total project expenditure. Dev pilots across 35 trials separately record $0.18207501, excluding Gemini judge charges. Engineering effort, local Docker compute, other development calls and all unknown usage are outside these totals. [4,9]

One-time optimization spending and recurring deployment spending answer different questions. If search and development cost S are amortized over N deployments, a cheaper selected configuration only saves money when S + N times C_selected is less than N times C_reference. We do not report a verified break-even count: S is incomplete and V's true cost is unknown. The measured reward gain over B0 also comes with higher recorded spend. Any deployment decision should therefore price additional reward, unknown-cost risk and engineering overhead together.

9. Failure classification and paired case studies

9.1 Classes and coverage

The deterministic classifier assigns a primary outcome from verifier evidence, trajectory exit status and events. Harness-level classes cover guardrail blocks, unparsable tool calls, budget exhaustion, unrecovered command timeouts and premature submission with missing outputs. Model-level classes cover wrong output and partial credit. A primary label is a diagnostic convention, not proof of a single cause. Archived infrastructure attempts are excluded. [8]

Outcome or class	Trials	Interpretation
Unparsable tool call	273	Harness protocol failure
Wrong output	49	Model-level result failure
Partial credit	16	Model-level incomplete result
Premature submit with missing output	15	Harness-level stopping failure
Guardrail blocked	7	Harness-level block preceding failure
Timeout without recovery	2	Harness-level recovery failure
Budget exhausted	0	No primary failures in this class
Unclassified	45	Evidence insufficient for a primary cause
Full success	31	Reward exactly 1

Table 7. Primary outcome classification across 438 scored trials

These outcomes cover 438 scored trials: 297 harness-level failures, 65 model-level outcomes short of full reward, 45 unclassified and 31 full successes. The classification covers all 11 configurations. Counts of tool-call failures should not be confused with all RepeatedFormatError exits, because the primary-label rules can assign another class first.

9.2 Representative trajectories

Harness-level failure. On B0/task 040, the agent must leave a Delta flight-search result page active. It produces three plain-text responses without a bash tool call. Stock mini-swe-agent returns format feedback and exits with RepeatedFormatError after the third consecutive invalid response. No command executes, and reward is 0. This illustrates a protocol boundary failure rather than evidence that the underlying browser task was impossible.

Model-level failure. On V-E/task 115, the agent must remove VS Code's default ctrl+f Explorer binding. It writes an empty keybindings.json, reads it back and

submits. The verifier expects a removal entry using command -list.find. The tool path and submission work, but the solution lacks the required domain semantics. Reward is 0. A generic execution guard cannot supply the missing VS Code knowledge.

Harness-level failure (harness-induced). On V/task 071, valid nested heredocs trigger U-FMT-03 five times. The agent later encounters missing packages, a missing conversion tool and a timeout. We report the false positive as a harness-induced failure, without claiming it alone caused the zero reward. H0 solves the task without that rule. Eval-phase defects informed v1.1 improvements but did not retroactively change the measured v1 outcome. [7,8,10]

9.3 Full harness wins and regressions against the default

Task 110, video orientation, provides a strict win. B0 issues ffmpeg commands but ends on repeated format errors and receives 0. V uses a vertical flip, handles output through the offload path, checks the saved file and submits, receiving 1. Its submission gate falsely treats home-relative paths as missing, yet the agent verifies the real file and succeeds. This shows both useful execution and a control defect within a successful trial.

Task 006, extracting gym and auditorium spaces from an OpenStudio model, shows the opposite reward direction. B0 inspects the command help, runs an OpenStudio Ruby script and writes a reduced model; the verifier awards 0.33 despite a later format-error exit. V writes a script but calls unsupported openstudio --script, then tries a ruby executable absent from the container, and terminates after format errors with reward 0. This is a partial-credit regression, not a full default pass followed by a harness failure.

Across V versus B0, six tasks improve, 33 tie and one regresses. There is no recorded task where B0 fully passes and V fails: B0's sole full pass is retained by V. Accordingly, no strict opposite-direction example exists. Task 006 is the closest observed regression and is labelled with its actual rewards. Both trajectories are directly traceable in Appendix A.

10. Discussion and generalization

10.1 What contributes to the gain

Prompt alignment gives the clearest standalone signal and greatly reduces calls on C0's paired subset. The stronger tool interface has a similar estimated reward gain

with higher cost. Budget awareness alone shows a small uncertain gain, while context and recovery controls do not demonstrate an independent benefit. Because C1 compaction never triggers, the experiment cannot evaluate that algorithm's effectiveness.

The full harness combines useful and potentially counterproductive changes. False-positive validation and path checks can interrupt otherwise legitimate plans. Several components also target the same plain-text termination problem, so their benefits need not add. We cannot infer that fixing a guardrail would necessarily improve reward: downstream plans may still be wrong. The case evidence supports targeted hypotheses for future testing, not a counterfactual corrected score.

10.2 Does the benefit shrink with a stronger model

The absolute point estimate is +0.075 for V minus B0 and +0.091 for V-E minus E. We see no observed shrinkage. However, the intervals overlap widely, and the V-E minus E interval includes zero. There is no statistical test here of the difference between those two treatment effects. A claim that the stronger model benefits more is therefore unsupported. The result is consistent with both models suffering from the same execution protocol, which the harness can improve before model capability becomes the limiting factor.

10.3 Held out results and development shift

The 40-task eval set is disjoint from the ten dev tasks used for model selection and development. The headline results and strict single-attempt pass rates in Section 6 are therefore held-out task outcomes for the frozen evaluated configurations. Post-eval v1.1 fixes are not counted as held-out gains because they use insights from the evaluated trajectories. Reusing these tasks for further tuning would turn them into development evidence and require another held-out set for an unbiased final claim.

For the stock pilots, mean reward falls from 0.200 on dev to 0.089 on eval for 20b, and from 0.366 to 0.169 for 120b. The descriptive gaps are approximately -0.111 and -0.197. Different task mixtures, live environments, corrected grader availability and single-run variation prevent attributing those gaps solely to overfitting.

11. Conclusions and limitations

Harness engineering materially improves the cheap stock baseline on the observed mean-reward metric. The full 20b harness approaches the expensive stock

model's reward while its recorded known inference cost is lower. The exact saving and formal equivalence remain unproven. The stronger model with the full harness achieves the highest reward at higher known cost. Prompt clarity gives the clearest standalone evidence, while other controls require careful evaluation because they can add cost, interfere with one another or reject valid work.

The evidence rests on one attempt per task, 40 eval tasks, 18 unknown-usage attempts, two missing C0 rewards and an automated classifier with 45 unclassified cases. Continuous verifier rewards differ from binary full completion. The sampled family balance differs from the full benchmark, and live websites and container dependencies can drift. H0 is a control rather than a proven equivalent of B0, C3 changes execution limits, and the component tests do not identify numeric sensitivities or full interaction effects.

The implementation and analysis are complete for this fixed-configuration study. The final lesson is to measure the execution harness and its own failure modes alongside model capability, while preserving the boundary between evaluated evidence and subsequent fixes.

12. Team contributions

Table 8 records each member's main contributions, drawn from the project's task allocation [2] and from the repository's commit history. Report sections are listed under their drafting owner; Ulfa Herdyani led the shared report and its final assembly.

Member	Main contributions	Report sections
Rohit Panda	Led infrastructure and integration: oracle gate, subset freeze, C0 prompt, all 11 harness configurations, multi-configuration runner, final evaluation runs and analysis outputs, and the dev false-positive review in the guardrails document. Took over running the analysis on the evaluation logs on 26 October (fix for archived attempts, lower-bound Pareto policy).	§1, §2, §11
Ghia Isha Kirti	Implemented the host-side guardrail rules and task-leak check with their tests, and the C2 (tool interface), C4 (recovery) and C5 (submission gate) components. Owns the failure taxonomy (labels and precedence order) that the failure classifier implements.	§4.2, §5
Kaivelikkal Abin Krishna	Built the harness core: Harbor agent, execution loop with hooks, container adapter, and OpenRouter gateway with call logging and telemetry. Implemented C1 (context) and C3 (budget), integrated the guardrails, and ran and documented the partial C1/C3 dev runs.	§3, §4.1
Syedyaseen Roshan Tushar	Wrote the deterministic failure classifier and its tests, tests for the export and run tooling, and the harness-v1.1 bug fixes and timeout handling made after the evaluation.	§7, §9.1, Appendix A
Ubaidulla Asmitha	Compiled the failure trajectory catalogue, the additions to the mapping of guardrail controls to course frameworks, and the guardrail evaluation review.	§9.2, §10, Appendix C
Ulfa Herdyani	Ran the dev model pilots and froze the model pair and price table. Wrote the cost collector, including handling of unknown token usage, and the analysis and Pareto-plotting scripts. Led the shared report and its assembly.	§6, §8, Appendix B

Table 8. Contributions by team member

References

[1] PE6203 Group Project 5 Cost-Aware Harness Optimization. Assignment brief in the repository root, Sections 2-4.

[2] Repository fact sheet and reporting plan. docs/report_tasks.md, reviewed 28 October 2026.

[3] Final eval protocol and incidents. results/eval/rohit-eval-pass1/README.md.

[4] Final per-trial and aggregate data. results/analysis/eval/rohit-eval-pass1/trials.csv and summary.csv. Strict pass counts and spending totals in this report are derived from these records.

[5] Paired comparisons and component ablations. results/analysis/eval/rohit-eval-pass1/comparisons.csv and ablations.csv.

[6] Harness design and integration contract. system_architecture.md and docs/harness_core.md. Model and harness YAML files under configs/.

[7] Safety rules and eval review. guardrails.md and docs/guardrail_eval_review.md.

[8] Failure outcomes and trajectories. results/analysis/eval/rohit-eval-pass1/failures/summary.csv, examples.md and docs/failure_catalog.md. Supporting discussion and example drafts are under docs/report_drafts/.

[9] Model pilots and selection. docs/model_selection.md, docs/model_pilot_summary.md and results/pilots/.

[10] Version separation and corrections. CLAUDE.md and docs/bug_fixes.md. harness-v1 contains evaluated behavior; harness-v1.1 contains subsequent fixes.

Repository: https://github.com/Roh00t/PE6203_Cost-Aware-Harness-Optimization

Appendix A Reproduction and evidence locations

The reported eval behavior is harness-v1 at 6c87103. The B0/E dependency snapshot pin is a runner environment correction documented separately. Current main includes later v1.1 behavior changes. Reproduction should therefore retain the evaluated harness and use the documented compatible runner, rather than assuming current main will reproduce identical trajectories. API responses and live websites also introduce variation.

The frozen subset is printed by scripts/task_list.py eval; its fingerprint is given in Section 2. Configurations are under configs/models/ and configs/harness/. Raw curated logs are under results/eval/rohit-eval-pass1/<config>/<task>/<trial>/. Harness trials include agent/calls.jsonl and events.jsonl; stock baselines include mini-swe-agent.trajectory.json. Verifier output and reward are in the trial's verifier directory. Archived attempts remain alongside the active evidence.

The following are the recorded eval and analysis command patterns. They document the existing experiment and do not instruct an additional paid run.

```
CONFIRM_PAID=1 CONFIRM_EVAL=1 uv run --frozen python
scripts/run_all_configs.py \
 --split eval --configs B0,E,H0,V,V-E \
 --run-name rohit-eval-pass1 --job-budget-usd 1.00 --keep-going
Pass 2 configurations: C5,C4,C1
Pass 3 configurations: C0,C2,C3
python3 scripts/export_run.py jobs/rohit-eval-pass1/<config> \
 results/eval/rohit-eval-pass1/<config>
uv run --frozen python scripts/analyze.py \
 --input results/eval/rohit-eval-pass1 --split eval \
 --output results/analysis/eval/rohit-eval-pass1
uv run --frozen python scripts/plot_pareto.py \
 --input results/analysis/eval/rohit-eval-pass1/analysis.json \
 --output results/analysis/eval/rohit-eval-pass1/pareto \
 --lower-bound-policy exclude
Repeat plotting with --lower-bound-policy include and a distinct
output stem.
```

Direct paired-case trial IDs are B0/006 __AJ39Ld3, V/006 __wJdvEmt, B0/110 __hkr5goh and V/110 __TTmwUyg. The full folder names prepend the task ID shown in Section 9.3. Figures 1-2 redraw the committed data in dollars per run, preserving lower-bound and incomplete-coverage labels. They do not add model executions or alter recorded rewards.

Appendix B Prices limits and strict outcomes

Model	Endpoint	Input USD per million	Output USD per million
GPT-OSS-20b	Darkbloom via OpenRouter	$0.018	$0.090
GPT-OSS-120b	Venice via OpenRouter	$0.030	$0.150

Table B 1. Frozen model endpoint prices

The price snapshot is 12 October 2026 and was frozen after the dev pilots. All prompt tokens use the listed input rate. The estimate does not apply provider-specific cache discounts. Verifier charges, including Gemini checks on dev task 009, are excluded from agent inference cost.

Configuration group	Task limit	Step limit	Task dollar cap	Command timeout
B0 E H0 and C0 C1 C2 C4 C5	2,400 s	None	None	30 s
H0+C3 and V	2,400 s	50	$0.0100	180 s
V-E	2,400 s	50	$0.0167	180 s

Table B 2. Per configuration execution limits

The per-run job ledger is a separate spending kill switch. Baselines retain stock settings rather than inheriting a C3 cap. A disabled step cap does not remove the Harbor task time limit.

Config	Full passes	Scored tasks	Strict pass at 1
B0	1	40	2.5 percent
E	4	40	10.0 percent
H0	2	40	5.0 percent
H0+C0	3	38	7.9 percent on scored tasks
H0+C1	1	40	2.5 percent
H0+C2	3	40	7.5 percent
H0+C3	3	40	7.5 percent
H0+C4	2	40	5.0 percent
H0+C5	3	40	7.5 percent
V	3	40	7.5 percent
V-E	6	40	15.0 percent

Table B 3. Strict single attempt outcomes for all configurations

C0's rate conditions on 38 scored tasks and is not a completed 40-task estimate. The two missing tasks are not silently treated as failures. Strict full passes differ from the mean-reward success metric used by the Project 5 brief.

Appendix C Framework mapping and control boundaries

Control	Course principle	Evidence and limitation
Disposable container and environment allow-list	Sandboxing and least privilege	Credentials stay outside task execution; containers still need network access
Grader-path and destructive-command checks	Execution authority and output validation	No G-rule matches in reviewed trajectories; latent rule gaps remain possible
C2 schemas and submission syntax	Validate before execution	Thirteen heredoc blocks were false positives in v1
Step cost and time limits	Budgets and explicit stopping	No primary failed-trial budget exhaustion label; C3 also lengthens command timeouts
C4 repeat and stall checks	Failure policy and bounded recovery	Intercepts may be harmful after state changes
C5 checklist and independent verifier	Completion checked in code	File existence is not semantic correctness; path extraction produced false positives
Calls events and full trajectories	Observability and accountability	Unknown usage is explicit; logs cannot recover unreported provider token counts
Frozen subset and version separation	Evaluation integrity and supply-chain control	v1.1 fixes cannot be claimed as measured v1 gains

Table C 1. Controls mapped to course principles

The mapping follows the repository's Week 7-12 course notes and guardrail mapping. It describes design intent and measured defects rather than certifying compliance with an external security standard. [6-8,10]

~*~